CIS 4270/5270: Trustworthy Machine Learning
Spring 2026: Course Project Draft Report

Team: Akshat Bokdia, Rashi Agrawal, Ritika Agarwal

Introduction

We propose a constraint-following recipe generation task. Given a list of available ingredients, the model must generate a quick (10-15 minute), student-friendly recipe using those ingredients. A **desirable recipe** should: use the provided ingredients appropriately, avoid unrelated ingredients, fit within the target cooking-time range, provide clear step-by-step instructions, and remain simple and practical for students.

Training Data

We generated **2000** synthetic data samples using GPT-4.1-mini. Each generated sample consists of an input ingredient list, a preferred recipe, and a rejected recipe. We let the model invent a plausible ingredient list using diverse seeds (cuisine, meal type, and dietary restrictions) to create diverse examples. Once the ingredient list is obtained, the pipeline then generates:

- a preferred recipe that satisfies the task constraints;
- a rejected recipe that intentionally violates one constraint, such as adding unavailable ingredients, taking too long, having too many steps, or requiring unrealistic equipment;
- an LLM-as-judge sanity check to verify that the preferred recipe satisfies the constraints before keeping the sample.

We then shuffle the accepted examples using a fixed random seed and split the dataset into **80% training (1,600 samples)** and **20% validation (400 samples)** sets. The resulting data is then formatted into separate files for different training methods:

- SFT: uses the input ingredient list and preferred recipe;
- DPO: uses the same input with a preferred and rejected output pair;
- RFT: uses the prompt plus reference fields needed for reward evaluation.

This gives us a shared synthetic data pool that supports SFT, DPO, and RFT experiments.

Evaluation Metrics

For each of the 400 validation samples, we evaluate generated recipes using both hard constraint metrics and soft quality metrics. Hard metrics measure whether the model follows explicit task requirements, while soft metrics capture overall usefulness and quality. The hard metrics measured are:

- **Ingredient Compliance Rate:** Uses provided ingredients appropriately and avoids unrelated ingredients.
- **Step Count in Range:** Recipe has the desired number of steps (3–7).
- **Cook Time in Range:** Recipe satisfies the quick cooking-time requirement.
- **All-Pass Rate:** Satisfies all three: ingredient, step-count, and cook-time constraints.

The soft metrics measured are:

- **Coherence Rate:** Recipe is clear, logical, and well-structured.
- **Student-Friendly Rate:** Recipe is practical and easy for students.

- **Average LLM Quality Score:** Overall score combining coherence and student-friendly rate.

We also report descriptive metrics to better understand output style:

- **Average Number of Steps:** Mean number of recipe steps across generations.
- **Average Cook Time:** Mean estimated cooking time across generations.

Base Model

Our base model is GPT-4.1-nano, evaluated in a zero-shot setting without task-specific fine-tuning. We used the same prompt format (system + user) written during data generation and separate file generation so that comparisons across the base model, SFT, DPO, and RFT models are fair.

Prompt

The system prompt was: *You are a recipe assistant for college students. Generate quick recipes (under 15 minutes) with 3-7 clear numbered steps, using only the ingredients provided.*

The user prompt was generated from each ingredient list using the following format: *Create a quick, student-friendly recipe using these ingredients: [ingredient list]. Keep it under 15 minutes with 3-7 clear steps.*

Inference configurations

For baseline evaluation, we used:

- **Base model:** GPT-4.1-nano (Temperature: 1.0, Max tokens: 1024)
- **Judge model:** GPT-4.1-mini (Temperature: 0.0, Max tokens: 512)

We used temperature 1.0 to allow natural recipe variation while keeping the prompt constraints fixed. The main baseline experiment tested whether prompt engineering alone was sufficient for constraint-following.

Base Model Results

Metric	Result
Ingredient compliance rate	99.25%
Steps in range	100.0%
Cook time in range	99.0%
All-pass rate	98.25%
Coherence rate	100.0%
Student-friendly rate	100.0%
Average LLM quality score	1.0

Avg. number of steps	6.12
Avg. cook time	10.69 min

The base model performs very strongly, achieving a 98.25% all-pass rate on the validation set. This suggests that GPT-4.1-nano already has substantial prior knowledge for simple, student-friendly recipe generation.
As a result, our fine-tuning experiments focus on whether task-specific training can preserve or improve this strong baseline while improving consistency and robustness across hyperparameter settings.

Supervised Fine-Tuning (SFT)

We generated SFT training data using GPT-4.1-mini. Each sample included:

- the recipe-generation prompt, including the ingredient list
- and a high-quality target recipe satisfying our constraints.

Hyperparameter Tuning and SFT Model Results

Given compute and cost constraints, we performed a six-configuration hyperparameter sweep over: number of epochs, batch size, and learning-rate multiplier.

Run	Hyperparameter Configuration	Final Train Loss	Final Valid Loss	Final Full Valid Loss	Num Steps
1	ep=1, bs=8, lr=0.5	0.373396	0.341913	0.350157	200
2	ep=2, bs=8, lr=0.5	0.285611	0.375854	0.337619	400
3	ep=3, bs=8, lr=0.5	0.216886	0.334872	0.341092	600
4	ep=2, bs=8, lr=0.2	0.325183	0.385566	0.335998	400
5	ep=2, bs=8, lr=1.0	0.282767	0.292470	0.344122	400
6	ep=2, bs=4, lr=0.5	0.331459	0.312745	0.339554	800

This sweep was designed to isolate the effects of training duration, optimization strength, and batch size.
Among the 6 runs, we chose Run 3 because of minimal final training loss and best accuracy and define it as SFT-alpha.
The SFT-alpha model performs comparably to the zero-shot base model. It exactly matches the base model's all-pass rate of 98.25%, while improving cook-time compliance from 99.0% to 100.0%. Ingredient compliance decreases slightly from 99.25% to 98.25%, suggesting that fine-tuning does not yet clearly outperform the strong baseline. Figure 1 summarizes the full comparison between the baseline and SFT-alpha across all reported evaluation metrics.

This result is still informative: the base GPT-4.1-nano model already shows near-ceiling performance on our prompt-engineered recipe task. Therefore, SFT improvements are expected to be small and may appear more in consistency, formatting, or robustness than in headline constraint-satisfaction metrics. In the final report, we will use the full six-run hyperparameter sweep to select the best SFT configuration based on validation loss and downstream evaluation metrics.

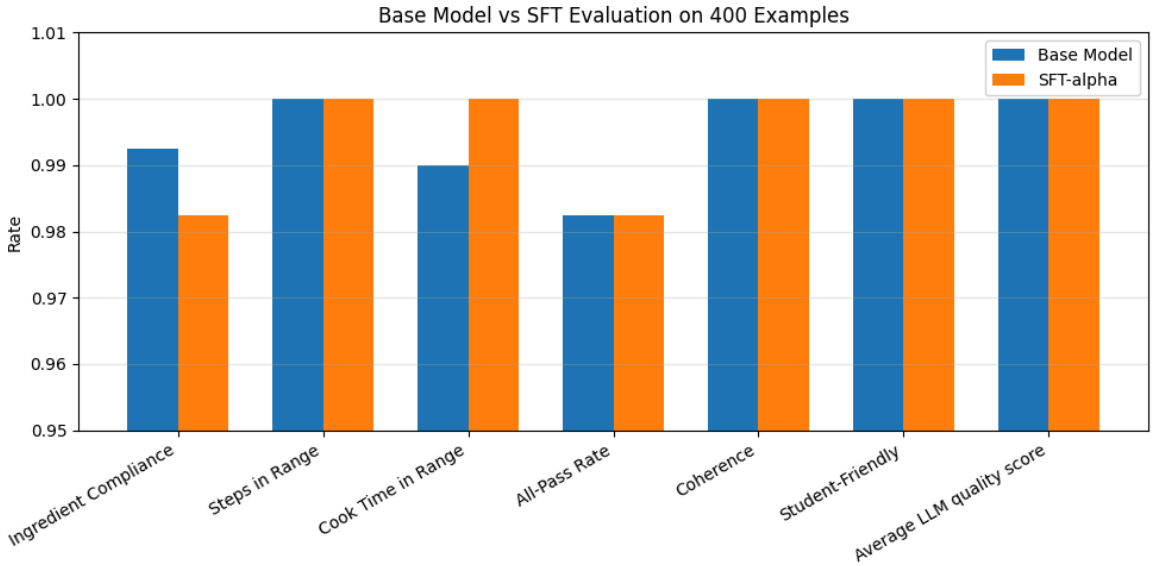

Figure 1. Comparison of zero-shot baseline and SFT-alpha on held-out evaluation metrics.